IERG3080 Tutorial 5

OO Review

Hoover H.F. Yin

Department of Information Engineering
The Chinese University of Hong Kong

October 19, 2016

Contact

- YIN Ho Fai, Hoover
- Office: Institute of Network Coding (INC), 7/F, SHB
- Email: yhf015 at ie dot cuhk dot edu dot hk

- Your local tutor
- Can speak Cantonese :D

Outline

OO

Object

Class

Inheritance

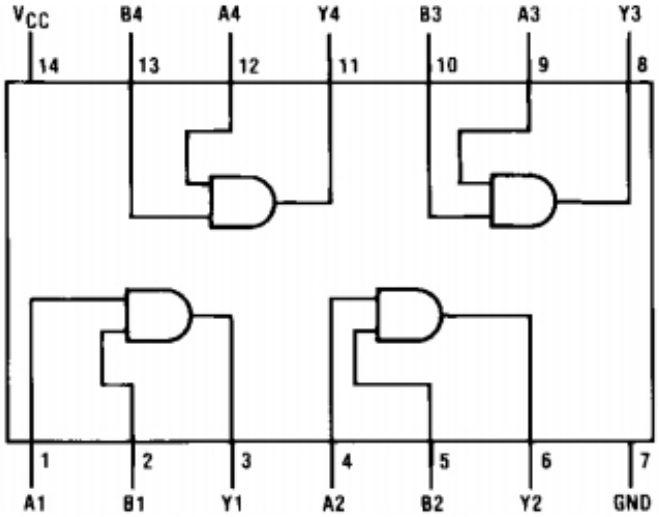
Misc.

Software Chip

Have you used those “hardware” chips before?

Source: http://www.electroschematics.com/8891/7408-datasheet/

Software Chip



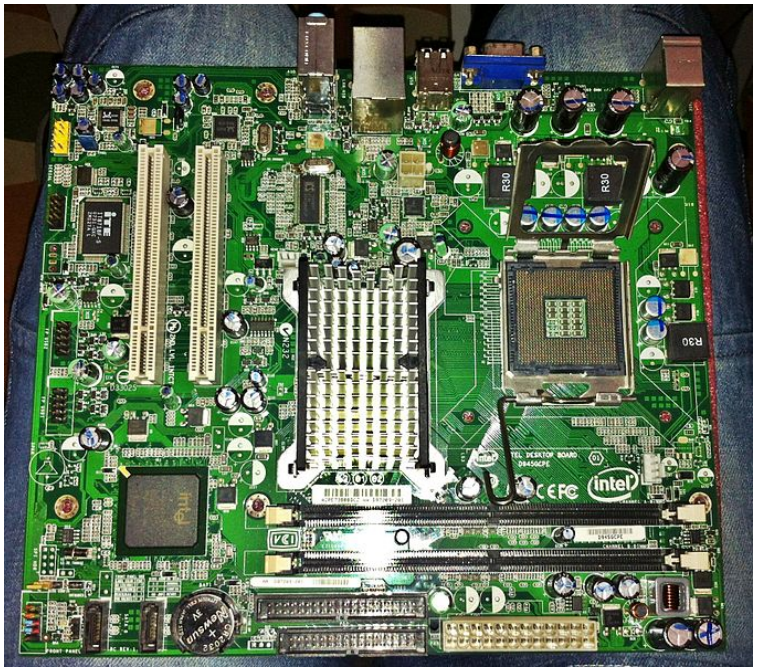
You only need to know

- What the chip can do
- How to use the chip

in order to use it, but not how it works

Source: http://www.electroschematics.com/8891/7408-datasheet/

Software Chip



You can have multiple instances of the same chip.
When you combine different chips (and other components), you can create a complex product

Source: https://en.wikipedia.org/wiki/Motherboard

Software Chip

Use this idea in software!

- You know what it can do
- You don't need to know why it works
- You can use it given the interface (pins assignment)
- Reusable – multiple instances

In a big project...

- Division of labour – everyone codes different chips
- But you need a design, e.g., how to use your chips

Problem Solving

Problem solving using a computer, you can design a solution which

- close to the machine
- close to the problem

Close to the machine?

- Design according to what the machine can do, e.g., C
- Run efficiently
- Machine friendly
- Good for small problem

Close to the problem?

- Design in the view of solving the problem, e.g., OO
- Not the most efficient way for the machine
- Human friendly
- Good for large problem

Object?

In real life, what is an object?

- Chair
- Car
- Sun
- Sand

How do you describe an object in real life?

Object?

How do you describe an object in real life?

- Method – What can it do?
  - Scissors – cut things
- Property – the features
  - My features – height: 1.x meter, weight: xyz kg
- Class – the model
  - e.g., iPhone 7 – same model but stored different data
- Inheritance – the same features for some classes
  - Humans and Monkeys – looks similar, intelligence
- Encapsulation – you can use it without knowing why it works
  - You don't know how the engine of a car works, but you can still drive

Object!

Example: My iPhone 6 Plus

- Manufacturer – Apple
- Model – iPhone 6 Plus
- Color – Space Grey
- Storage – 128GB

What it can do?

- Make a phone call
- Install Apps
- Apple Pay

Object!

In the syntax of OOP...

```
//properties
myPhone.manufacturer = "Apple";
myPhone.model = "iPhone 6 Plus";
myPhone.color = "Space Grey";
myPhone.storage = 128;

//methods
myPhone.Call();
myPhone.InstallApps();
myPhone.ApplePay();
```

Class vs. Object

Something wrong?

- Human skipped the lecture.

This seems make sense

- Siu Ming skipped the lecture.

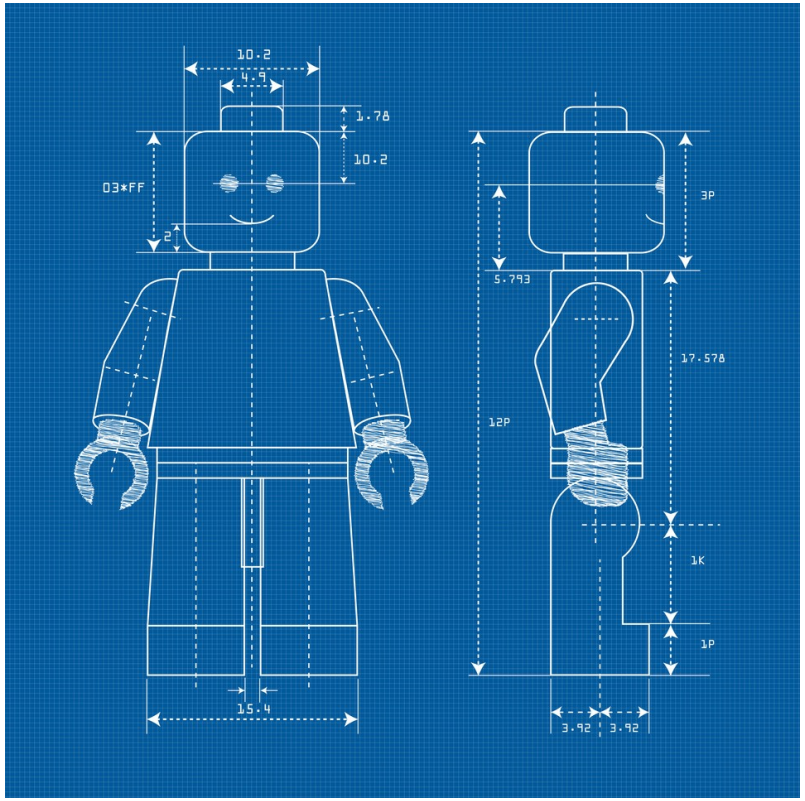
So what's wrong with the first sentence?

- Siu Ming is a human
- "Human" does not correspond to a specific individual

Human is a class, Siu Ming is an object

- Class is a blueprint
- Object is an individual

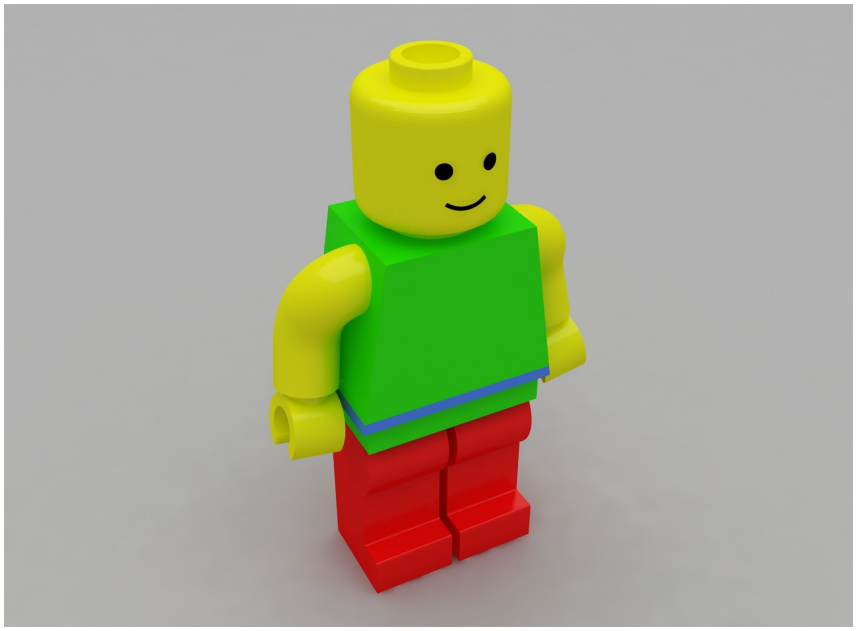
Class vs. Object



This blueprint specifies how a Lego human looks like, but you don't have a Lego human on hand!
This IS a class.

Source: https://crated.com/art/141810/lego-blueprint-by-alessandroaru

Class vs. Object



According to the blueprint, you can make the Lego human. As much as you want.
This IS an object. You can have multiple objects according to the same class.

Source: http://www.turbosquid.com/3d-models/lego-human-3d-model/711309

Terminology

In a nutshell...

- Field – The variables in the object
- Property – Access by the syntax like `myPhone.model`
- Method – Functions, e.g., `myPhone.Call()`
- Modifier – Access permission

Getter/Setter

Make everything under your control... Users are evil!!
I won't allow someone to state my iPhone model is Note 7!!

Well??? What does it mean?

```
public int favNum { get; set; }
```

- Auto property – the compiler creates the field for you
- Easy to read – it's a property for sure
- Easy to maintain – what if you change the spec?
  - It is a trouble to covert a field in to a property!
    - Although the syntax are the same, but accessing a field is accessing a variable, while using getter/setter is calling functions!
    - You need you compile other codes again after the change although the codes are the same

Create an object

```
Random rand = new Random();
Random rand2 = new Random(1999);
```

When you `new` an object... The program does the following

- Allocate a memory space for the object
- Initialize the fields
- Run the constructor

Which constructor? It is specified after your `new` operator.

Constructor

```
public class MyClass
{
    public MyClass(){} // the default constructor
}
```

- The compiler will make a default constructor for you if...
  - you have not coded any custom constructor
- Constructor is not a method, although it looks like a function
- No return type indicated
- The same name as the class name

Inheritance

What a smart phone can do?

- Make a phone call
- Install Apps

```
public class SmartPhone
{
    public bool Call(string no){}
    public bool InstallApps(int appID){}
}
```

Different smart phone has different extra functions

Inheritance

So we use inheritance to indicate the extra functions

```
public class iPhone : SmartPhone
{
    public bool ApplePay(){}
}

public class Note7 : SmartPhone
{
    public void Explode(){}
}
```

Before running its own constructor, it will first run the constructor of its parent class.

Override

How about there's a default behaviour? Only some of the derived classes have different behaviours? Mark it `virtual` then `override` it!

```
public class Human
{
    virtual public void Sleep(){
        Console.WriteLine("In the bedroom");
    }
}

public class Student : Human
{
    override public void Sleep(){
        Console.WriteLine("In the classroom");
    }
}
```

Abstract Class

```
public class Animal
{
    public void Talk(){}
}
```

Different `Animal` talks in different way...

- We have a default behavior if we use `virtual`
- How to make sure the derived classes implement this method?

Abstract Class

```
abstract public class Animal
{
    abstract public void Talk();
}

public class Cat : Animal
{
    override public void Talk(){
        Console.WriteLine("Meow");
    }
}
```

Interface

Interface is a contract. Once you promise to implement the interface, you MUST provide the services (implement the methods). As it tells the others how to use your class, of course the methods in the interface are public.

- `Rectangle` – `width`, `height`
- `Circle` – `radius`
- Not so alike but both should provide the method `Area()`

```
public interface Shape
{
    double Area();
}
```

Interface

```
public class Circle : Shape
{
    public double radius {get;set;}
    public double Area(){return radius*radius*Math.PI;}
}

public class Rectangle : Shape
{
    public double width {get;set;}
    public double height {get;set;}
    public double Area(){return width*height;}
}
```

static

- `Math.Sqrt()`
  - Hey, `Math` is a class
  - Do you need to create an object?

This is how this method declared:

- `public static extern double Sqrt(double d);`

This is a class method. You don't need to create an object before you use it!

Generic Programming

Suppose I have the following swap function for `int`

```
public static void swap(ref int a, ref int b)
{
    int tmp;
    tmp = a;
    a = b;
    b = tmp;
}
```

If I want to have a swap function for `double`, why I need to copy the whole method but only changing the data types???

Generic Programming

Remember that < >? You used that before! (e.g., List<int>)

```
public static void swap<T>(ref T a, ref T b)
{
    T tmp;
    tmp = a;
    a = b;
    b = tmp;
}
```

We can then use swap<int>(a,b) and swap<double>(a,b) and don't need to copy the codes :D